

FSM 리팩터링 변경 정리

Assets/Scripts/FSM 작업 트리의 Git diff와 현재 소스 기준.

상태 객체 중심 구조를 TransitionRule 중심 구조로 전환하는 진행 상황을 기록합니다.

진행 중

코드 기준 검토

검증 전

문서 범위

- 변경 대상: Assets/Scripts/FSM/
- 비교 기준: 현재 작업 트리와 Git 기준점
- 변경 규모: 추적 파일 기준 40개 변경, 256줄 추가, 674줄 삭제. TransitionRules 폴더는 신규 미추적 파일로 포함.
- 목적: State가 Controller를 직접 조작하던 구조를 줄이고, 상태 전이 조건을 규칙 객체로 분리.

핵심 결론

상태 동작은 각 State에, 상태 전이 판단은 TransitionRule에 두는 방향으로 구조가 바뀌었습니다. 다만 Factory의 전이 등록, 전이 이벤트 연결, 구독 해제, 컴파일 정합성은 아직 완료되지 않았습니다.

1. 변경 요약

리팩터링은 상속 깊이와 Controller 의존도를 줄이는 데 집중되어 있습니다.

영역	이전	현재 변경	상태
상태 기반	AgentStateBase<T> + GroundedAgentStateBase	비제네릭 AgentStateBase 하나로 통합	적용
상태 전환	State 내부에서 _agent.ChangeState 직접 호출	ITransitionRule을 순차 평가하고 OnTransition 발행	부분 적용
상태 머신	AgentStateMachine<IAgentState>	AgentController가 _currentState를 직접 보유	적용
의존성	State가 AgentController 전체를 주입받음	Animator / Handler / Input을 생성자로 주입	적용
입력/체력	Handler가 Controller 메서드 호출	Factory에서 Rule로 연결하려는 구조	미완료
애니메이션 이벤트	현재 State.OnAnimationEvent 호출	Controller가 상태별 이벤트를 분배하고 Action 발행	부분 적용

의도된 실행 흐름

Input / Health / Animation Event → TransitionRule의 플러그 또는 조건 → AgentStateBase.ShouldTransition() → OnTransition(StateType) → AgentController.ChangeState()

현재 소스에서는 마지막 두 연결(OnTransition 구독 및 Factory 전이 등록)이 완성되지 않아, 이 흐름은 설계 방향이지 완성된 실행 경로는 아닙니다.

2. 이전 구조 → 바뀐 구조

클래스별 책임 이동을 중심으로 비교합니다.

클래스 / 영역	이전 책임	현재 책임	효과
AgentStateBase<T>	Controller 제네릭을 보관하고 Enter/Execute/FixedExecute/입력.애니메이션 이벤트 제공	AgentStateBase로 단순화. Enter → 규칙 구독, Execute → 규칙 평가 후 OnExecute	State 동작과 전이 조건의 분리 기반
GroundedAgentStateBase	GroundedAgentController 전용 중간 상속 계층	삭제	상속 단계 축소
Idle / Move	_agent.IsIdle 검사 후 직접 ChangeState	애니메이션 및 이동 Handler만 수행	MoveInputTransition으로 외부화 예정
Jump / Fall	착지-시간-애니메이션 종료를 직접 검사하고 직접 전이	점프 / 공중 이동 동작만 유지	Land / JumpFall 등의 Rule로 외부화
Attack / Hit / Death	State가 종료 플래그를 직접 기록하고 Controller 호출	애니메이션 상태 및 Handler 조작에 집중	종료 이벤트 Rule 추가 필요
AgentController	AgentStateMachine, Health/Input Handler, 공통 동작 메서드 소유	현재 State 직접 보유, ChangeState 직접 수행, 애니메이션 이벤트 분배	중간 머신 제거, 단 전이 브리지 필요
Player / Monster Factory	Controller 전체를 전달해 State 생성	FactoryData로 필요한 의존성만 전달	테스트-재사용성 개선

상태의 역할 변화

이전: State = 동작 + 조건 검사 + ChangeState 호출

현재 목표: State = Enter / OnExecute / Exit 동작, Rule = 전이 조건, Controller = 실제 상태 교체

3. State 계층 변경 상세

상태별로 제거된 책임과 남은 동작을 정리합니다.

State	제거 / 이동된 코드	현재 남은 역할	추가할 Rule
IdleState	IsIdle 검사 및 Move 전이	Idle 애니메이션, 정지 처리	MoveInputTransition, AttackTransition, JumpTransition, GroundedFallTransition
MoveState	IsIdle 검사 및 Idle 전이	Move 애니메이션, MovementHandler.HandleMove	StopMoveTransition, AttackTransition, JumpTransition, GroundedFallTransition
JumpState	점프 타이머, 착지/낙하 판단, 입력 이벤트	Jump 애니메이션, HandleJump, 공중 이동	LandTransition, JumpFallTransition, AirAttackTransition
FallState	착지 후 Idle/Move 선택, 공격 입력 이벤트	Fall 애니메이션, HandleAirMove	LandTransition, AirAttackTransition
AttackState	공격 완료 플래그 / State 직접 전이	공격 애니메이션과 공격 타입 설정	AttackEndTransition
HitState	_isHitFinished 및 직접 Idle 전이	피격 애니메이션, 공격 타입 초기화	GetHitEndTransition
DeathState	애니메이션 종료 처리	사망 애니메이션, 이동 중단	DeathEndAction 또는 별도 종료 처리

주의: 동작 호출 위치

기존 `FixedExecute()`가 삭제되었으므로 이동 처리의 Update / FixedUpdate 책임을 의도적으로 재배치해야 합니다. 현재 GroundedAgentController.FixedUpdate()는 GroundDetector 갱신만 수행합니다. Rigidbody2D 기반 이동이라면 Handler 내부 처리와 물리 프레임 일관성을 별도로 검증해야 합니다.

4. 삭제된 부분과 추가된 부분

Git 기준점 대비 주요 파일 단위 변경입니다.

삭제 또는 제거

항목	변경 이유 / 의미
IAgentState 및 InputKeyType	State 인터페이스에서 OnInputEvent / OnAnimationEvent 기반 전이 경로 제거
AgentStateMachine<IAgentState>	Controller가 현재 State를 직접 보유하도록 전환
AgentSignals / 기존 ITransitionRule	초기 제안형 신호·우선순위 모델을 제거하고 새 TransitionRules 폴더 구조로 재정리
AgentHealthHandler	HP 변화 → Controller.OnHit/OnDeath 직접 호출 경로 제거
GroundedAgentStateBase	Grounded 전용 중간 상속 계층 제거
GroundedIdleState / GroundedMoveState	Idle/Move 공통 상태로 통합하고 지상 조건을 Rule로 이전
GroundedAgentInputHandler / 분리 JumpInput 인터페이스	점프 입력 인터페이스를 Agent/Input/IAgentInput.cs로 통합

추가

항목	역할
StateFactoryData / PlayerStateFactoryData / MonsterStateFactoryData	Controller 전체 대신 상태 생성에 필요한 의존성을 묶어 전달
TransitionRules 폴더	Attack, Jump, Land, JumpFall, GetHit, Death, GetHitEnd 규칙의 시작점
ITransitionRule / IEventTransitionRule	폴링 규칙과 구독형 이벤트 규칙을 구분
AgentStateBase.OnTransition	Rule이 결정한 StateType을 State 외부로 알리는 전이 요청 채널
AgentController.OnAnimationEnded	HitState의 애니메이션 종료를 이벤트 Rule에 전달하기 위한 event source

5. 새 전이 구조와 이벤트 구독

ITransitionRule은 매 프레임 평가되고, IEventTransitionRule은 상태 생명주기에 맞춰 이벤트를 수신해야 합니다.

권장 흐름

```
State.Enter() → 해당 State의 IEventTransitionRule.Subscribe()
State.Execute(dt) → 등록 순서대로 ShouldTransition(dt) 평가 → 참이면 OnTransition(NextState)
AgentController.ChangeState(next) → 이전 State의 모든 이벤트 규칙 Unsubscribe() → 다음 State.Enter()
```

현재 AgentStateBase는 `Enter()`와 `AddTransition()` 양쪽에서 Subscribe()를 호출하며, `Exit()`에서는 전체 Unsubscribe()를 수행하지 않습니다. 따라서 중복 구독 및 종료 후 이벤트 수신 가능성이 있습니다. 구독 책임은 Enter/Exit 한 쌍으로 단일화해야 합니다.

GetHitEndTransition의 적합한 사용

`AgentController.OnAnimationEvent(End)`가 현재 State가 HitState일 때만 `OnAnimationEnded`를 발생시키고, `GetHitEndTransition`이 이를 구독하는 구조는 적절합니다. 단, HitState에 Rule을 등록하고 HitState 종료 시 반드시 구독을 해제해야 합니다.

```
HitState → GetHitEndTransition(AgentController) → OnAnimationEnded event → shouldTransition = true → Idle 또는 Move
```

UniRx 기반 체력 규칙

GetHitTransition / DeathTransition은 `Subscribe()` 반환값(IDisposable)을 보관해 `Unsubscribe()`에서 Dispose()해야 합니다. 현재 파일의 `if (CurrentHealth != null) return;` 및 `if (IsDead != null) return;`는 조건이 반대여서 정상 주입 시 구독이 시작되지 않습니다.

6. 현재 확인된 연결 누락 및 검증 항목

아래 항목은 diff와 현재 소스의 정적 대조 결과입니다. Unity 컴파일 및 플레이 검증은 아직 실행하지 않았습니다.

우선	확인 항목	영향	권장 조치
P0	OnTransition += ChangeState 연결이 없음	Rule이 참이어도 상태가 교체되지 않음	Factory 생성 후 모든 State의 OnTransition을 controller.ChangeState에 연결
P0	PlayerStateFactoryData에 CombatHandler가 없는데 Factory는 data.CombatHandler 사용	컴파일 오류	Data 정의 또는 Factory 생성자 인자를 정합화
P0	PlayerController FactoryData에 CombatInput / JumpInput이 전달되지 않음	Attack/Jump Rule에서 null 가능	CombatInput, JumpInput 명시 주입
P0	GetHitTransition / DeathTransition의 null guard가 반대	구독이 시작되지 않음	IDisposable 보관 + subscription null 기준으로 수정
P1	AddTransition과 Enter에서 중복 Subscribe	중복 콜백 / 메모리 누수 위험	구독은 Enter, 해제는 Exit로 통일
P1	AttackTransition은 입력을 누르는 동안 true	공격 종료 후 재진입 가능	입력 소비 또는 edge-trigger / request 방식 적용
P1	Idle에만 Attack / Jump Rule 등록	Move/Jump/Fall 상태에서 허용 규칙 누락	State별 규칙 매트릭스 구성
P2	TransitionRules의 등록 순서가 우선순위	동시 조건 충돌 시 결과가 암묵적	등록 순서 규약 문서화: Death > Hit > Action > Movement

권장 최소 검증

- EditMode: 각 Rule의 ShouldTransition 반환값과 이벤트 구독/해제를 단위 테스트
- PlayMode: Idle ↔ Move, Grounded → Jump → Fall → Land, Hit → End → Idle/Move, Death 우선 전이
- 콘솔: 컴파일 오류 0개, 전이마다 Enter/Exit가 정확히 한 번씩 호출되는지 로그 확인

7. 예정사항 / 적용 순서

요청된 후속 작업을 위험도와 의존성 기준으로 정렬했습니다.

순서	예정 작업	완료 기준
1	AgentStateFactory를 상속 기반 공통 Factory로 정리	PlayerStateFactory / MonsterStateFactory가 공통 생성 흐름과 FactoryData 계약을 공유
2	FactoryData 정합화	Animator, MovementHandler, MovementInput, CombatInput, CombatHandler, GroundDetector, JumpInput의 필요 범위를 명확히 전달
3	PlayerStateFactory에서 State 생성 후 Rule 등록	각 State에 필요한 TransitionRule을 등록하고 OnTransition을 controller.ChangeState에 연결
4	기본 이동 Rule 추가	MoveInputTransition, StopMoveTransition, GroundedFallTransition, LandTransition 적용
5	액션 Rule 추가	AttackTransition, JumpTransition, JumpFallTransition, GetHitTransition, DeathTransition 적용
6	애니메이션 완료 Rule 추가	GetHitEndTransition 및 AttackEndTransition으로 State 종료 처리
7	구독 수명 정리	IEventTransitionRule의 Subscribe/Unsubscribe가 State Enter/Exit에서 정확히 한 번씩 발생
8	검증 및 물리 조정	플레이 테스트, 입력 홀드 재진입 방지, 공중 이동/플랫폼 가장자리 이슈 재확인

PlayerStateFactory의 권장 전이 매트릭스

Idle: MoveInput / Attack / Jump / GroundedFall / Hit / Death
Move: StopMove / Attack / Jump / GroundedFall / Hit / Death
Jump: Land / JumpFall / AirAttack / Hit / Death
Fall: Land / AirAttack / Hit / Death
Attack: AttackEnd / Hit / Death
Hit: GetHitEnd / Death
Death: 종료 Action

8. 권장 설계 원칙

이번 리팩터링의 방향을 유지하면서 안정화하기 위한 기준입니다.

1. State는 동작만 가진다

State는 애니메이션 bool, 이동 Handler, 공격 Handler 등 현재 상태에서 수행해야 할 동작만 책임집니다. 다른 State로 갈지 판단하거나 Controller를 직접 호출하지 않습니다.

2. Rule은 조건만 가진다

Rule은 입력, GroundDetector, Health, Animation Event 같은 읽기 전용 의존성을 받아 `ShouldTransition`만 결정합니다. 상태 변경이나 공격 실행처럼 부작용을 넣지 않습니다.

3. Controller는 전이 실행만 가진다

AgentController는 State가 발행한 StateType을 받아 Exit → 현재 State 교체 → Enter 순서로 실행합니다. 전이 우선순위와 등록은 Factory가 소유합니다.

4. 이벤트는 상태 수명과 함께 해제한다

GetHitEndTransition처럼 상태 전용 이벤트는 Enter에서 구독하고 Exit에서 해제합니다. Agent 생명주기 전체에 필요한 Health/Death 이벤트는 Controller 또는 전역 Rule 집합이 소유하고 OnDestroy에서 해제합니다.

결론

현재 변경은 상속 구조 축소와 의존성 명시화는 상당 부분 완료됐습니다. 다음 단계의 핵심은 Rule을 늘리는 것이 아니라, Factory의 등록·Controller의 전이 실행·이벤트 구독 수명을 하나의 일관된 경로로 완성하는 것입니다.